

Tarea 2:

Application Profiling with Score-P and Cube

Jose E. Flores Q.
Escuela de Ingeniería Eléctrica
B82994
SP-2136
II-24

Abstract—En el presente documento se resume la presentación dada por el Dr. Ing. Bernd Mohr del Jülich Supercomputing Centre titulada *Application Profiling with Score-P and Cube*. En ella, se comienza por una pequeña introducción al análisis de rendimiento de programas, en específico programas paralelos, al igual que a las herramientas disponibles para realizarlo. Luego, se introduce la herramienta de análisis de rendimiento *Score-P*, la cual se define como una *infraestructura de medición de rendimiento escalable para código en paralelo*. Esta infraestructura consiste de la interacción de múltiples herramientas para análisis de rendimiento. Durante la presentación se mencionan dichas herramientas y se procede a entrar más en detalle sobre la herramienta llamada *Cube*. Esta herramienta permite la visualización de el análisis de perfilamiento, o *profiling*, lo cual facilita el estudio del rendimiento de un programa en específico.

I. HERRAMIENTAS DE RENDIMIENTO PARALELO

El presentador inicia dando un contexto sobre el análisis de rendimiento en programas en paralelo, en específico acerca de lo que ofrecen las herramientas que existen para poder realizar esta tarea. Se habla acerca de que es posible dividir el análisis de rendimiento en dos clases: analítico y experimental. El tipo analítico consiste en el desarrollo y estudio de modelos para el análisis de rendimiento, mientras que el experimental consiste en, como su nombre lo indica, realizar experimentos sobre el código y obtener conclusiones de esa forma. El enfoque principal de la presentación es este último tipo de análisis y es en el que se basan las herramientas introducidas.

A. Análisis Experimental

Se procede a comentar acerca de la manera en la que el análisis experimental de rendimiento se lleva a cabo, y la naturaleza cíclica de este proceso. La primera parte importante del proceso de análisis consiste en la *instrumentación*. Esta idea se refiere a las herramientas o “instrumentos” utilizadas para realizar las mediciones necesarias para llevar a cabo el análisis. Estos pueden ser ya sea el usuario, o líneas de código extra (como *probes* o *hooks*). La siguiente parte del proceso consiste en la *medición*. Esta parte se encarga de recolectar la información relevante para el análisis (se hablará más al respecto adelante). Luego, está el *análisis*. Una vez obtenidas las mediciones necesarias, se procede a analizar dichos datos para llegar a conclusiones sobre el rendimiento del programa y tomar decisiones al respecto. Después de haber realizado el análisis y llegado a conclusiones, prosigue la parte de

presentación. El propósito de esta parte es tomar los resultados obtenidos y hacerlos legibles para los humanos para poder ser presentados. Finalmente, está el paso de *optimización*, en el cual se intenta eliminar los problemas encontrados durante el análisis. Esta parte usualmente se le deja al usuario.

Este ciclo se repite hasta que se acabe el tiempo disponible o hasta que, ojalá, se llegue a un estado satisfactorio del rendimiento del programa.

B. Medición de Rendimiento

Con respecto a la medición de rendimiento, hay dos preguntas fundamentales: *¿cuándo?* y *¿cómo?*

La pregunta de *cuándo* se puede responder con *disparadores externos* y *disparadores internos*. Los externos corresponden a componentes asincrónicos fuera del código, como contadores de hardware, que disparan la toma de muestras. El método más utilizado para este tipo se conoce como *sampling*. Por el otro lado, los disparadores internos son componentes sincrónicos dentro del mismo código. Un ejemplo de estos es la instrumentación del código mencionada previamente.

De igual manera, la pregunta de *cómo* se puede responder con dos respuestas. La primera es conocida como perfilamiento o *profiling*, y consiste de la suma de eventos (llamadas a funciones, paso de mensajes, etc) en el tiempo. En contraste, el *tracing* consiste en una secuencia de estos eventos en el tiempo.

1) *Profiling*: El perfilamiento, o *profiling* en inglés, se trata del registro de información agregada en el tiempo. Ejemplos de esta información podrían ser el tiempo o contadores. Este registro debe llevarse sobre entidades del programa y el sistema. Es decir, el registro de información agregada se lleva a cabo sobre funciones, bucles, procesos, hilos, etc. Además, parte del perfilamiento consiste en obtener información estadística sobre la información recopilada (mínimo, máximo, media...).

Algunos problemas que surgen con este método de análisis son lo que se conoce como “cuellos de botella desplazantes entre procesadores”. Esto quiere decir que, de vez en cuando, existen cuellos de botella que se van desplazando o moviendo de procesador en procesador lo cual provoca que los desbalances de carga se “cancelen” y estos no se vean reflejados en los resultados obtenidos. También, es posible que el desbalanceo de carga cambie con el tiempo, y ocurra lo mismo.

2) *Tracing*: El método de trazado, o *tracing* en inglés, consiste en el registro de información acerca de eventos significativos. Dentro de estos “eventos significativos” se encuentran la entrada y salida de alguna región de código (función, bucle, etc) y la transmisión y recepción de mensajes. Una parte importante del trazado consiste en el registro de metadatos sobre los datos recopilados, como por ejemplo fecha, hora o información específica al evento. Una vez realizado lo anterior es posible obtener lo que se conoce como trazo de eventos, o *event trace*, el cual consiste en el ordenamiento de eventos cronológicamente y permite obtener una imagen del comportamiento en el tiempo de nuestro programa o sistema.

Ventajas de este método son que es posible reconstruir el comportamiento dinámico del sistema a través de las trazas. Del mismo modo, también es posible obtener un perfil del programa por medio de las trazas. Una desventaja es que para archivos de traza muy grandes, solo es posible utilizarlos en configuraciones pequeñas o por períodos cortos de tiempo.

Cabe recalcar que no existe una única herramienta adecuada para todo problema, y que los requerimientos y restricciones de este son los que al final determinarán la escogencia óptima de las herramientas que se utilizarán para realizar el análisis.

II. ECOSISTEMA SCORE-P

El ecosistema *Score-P* es un conjunto de herramientas que permiten la instrumentación de aplicaciones al igual que la medición de los datos relevantes al rendimiento. El ecosistema utiliza diferentes herramientas para abarcar el alcance completo que ofrece, como *PAPI* para la utilización de contadores de hardware. Los perfiles realizados mediante *Score-P* son exportados en formato *Cube4* y las trazas en formato *Open Trace Format (OTF2)*.

Algunas de las herramientas de las cuales se compone el ecosistema completo de *Score-P* son las siguientes:

- **Cube**: Utilizada para visualizar de forma gráfica el perfilamiento realizado.
- **Tau ParaProf**: Otra herramienta para la visualización de perfiles.
- **Tau PerfExplorer**: Base de datos que permite comparar múltiples experimentos.
- **Scalasca**: Análisis automático para generar un reporte mejorado.
- **Vampir**: Herramienta para visualizar trazas gráficamente.
- **Extra-P**: Modelado de las mediciones para realizar predicciones sobre el comportamiento del sistema/programa.

III. MEDICIÓN DE PERFIL DE APLICACIONES PARALELAS CON SCORE-P

A. *Score-P*

Score-P consiste en una “infraestructura escalable de medición de rendimiento para código en paralelo” la cual es de código abierto y mantenida por una comunidad de colaboradores y usuarios.

Score-P ofrece instrumentación del código para los diferentes paradigmas de programación en paralelo: Multi-processing (i.e. MPI), Multi-threading (i.e. OpenMP), Aceleradores (i.e. CUDA), o cualquier combinación de estos. Además *Score-P* ofrece mediciones flexibles sin recompilación, tanto para *profiling* como para *tracing*. Otra de las ofertas de esta herramienta es la funcionalidad de entrada/salida (I/O), la cual es altamente escalable.

B. Flujo de Trabajo del Análisis de Rendimiento

El flujo de trabajo del análisis de rendimiento utilizando *Score-P* consiste de tres partes: *instrumentación*, *medición* y *análisis*.

En la parte de la instrumentación se tiene lo que son el código fuente, el instrumentador (compilador/enlazador), al igual que el ejecutable instrumentado. Luego, en la parte de medición se tiene la aplicación instrumentada, las librerías de medición las librerías de contadores de hardware (*PAPI*) y un perfil de resumen. Finalmente, en la parte de análisis, se tiene lo que es la manipulación del perfil, el perfil mejorado y el buscador de perfiles.

IV. ANÁLISIS DE PERFIL CON CUBE

Cube es una herramienta que permite perfilar y visualizar de manera gráfica los perfiles de las aplicaciones en paralelo. La razón de su nombre viene del hecho de que se basa en una medición en 3 dimensiones. Estas dimensiones corresponden a *métricas*, *árbol de llamadas* y *ubicación del sistema*.

Basándose en lo anterior, esta herramienta permite descomponer el programa en sus componentes básicos, y brindar información acerca de las tres dimensiones mencionadas anteriormente en cada uno de esos componentes de manera visual. Lo anterior permite encontrar problemas o cuellos de botella en nuestro código que de otra manera sería muy difícil de identificar.

V. CONCLUSIÓN

En conclusión, el análisis de rendimiento de programas paralelos es un proceso iterativo que requiere la instrumentación, medición, análisis, presentación y optimización del código. Las herramientas presentadas, en particular *Score-P* y *Cube*, facilitan este proceso permitiendo instrumentar y recolectar datos de rendimiento de aplicaciones paralelas sin necesidad de recompilar. *Score-P* es una infraestructura escalable que admite diferentes paradigmas de programación paralela, y su ecosistema incluye herramientas que abarcan desde la instrumentación de código hasta la visualización y análisis automático de datos.

Por otro lado, *Cube* destaca por su capacidad para representar gráficamente los perfiles de rendimiento, permitiendo descomponer los programas en tres dimensiones: métricas, árbol de llamadas y ubicación del sistema. Esto facilita la identificación de cuellos de botella y problemas de rendimiento que de otro modo serían difíciles de detectar. En resumen, estas herramientas proporcionan un conjunto completo de

soluciones para el análisis y mejora del rendimiento en aplicaciones paralelas, contribuyendo así a optimizar la eficiencia y escalabilidad del software.

REFERENCES

- [1] Bernd Mohr, *Application Profiling with Score-P and Cube*, Jülich Supercomputing Centre, 2022. Available: https://www.youtube.com/watch?v=3HtR89L2u9Q&list=PLhznUsiowt26GyqmTTO1joujbeSg2_WHK.